

Project Specification: Trip Expense Tracker App V2

This document outlines the comprehensive architecture, feature set, and development roadmap for a web application designed to simplify group trip finances, with offline capabilities and a Progressive Web App (PWA) distribution model.

1. Core Architecture & Features

1.1 The Invite System (Frictionless Onboarding)

To eliminate the need for complex email invitations, trips are protected and accessed via a randomly generated 6-digit alphanumeric PIN (e.g., A9B2X7). Users simply input this PIN on the "Join Trip" screen to be authenticated and added to the participant array.

1.2 The Kitty System (Centralized Fund)

The "Kitty" acts as a centralized digital wallet for the trip.

- **Group Lead:** Only the designated Group Lead (creator of the trip) has write-access to log expenses against the Kitty.
- **Prefunding:** Participants transfer a fixed amount (e.g., ₹5,000) to the Group Lead prior to the trip. The Group Lead logs these deposits, creating the initial Kitty Balance.
- **Spending:** Group dinners, cabs, and hotel bookings are logged by the Group Lead as Kitty expenses, automatically deducting from the centralized balance and recalculating per-person shares.

1.3 Personal Pockets (Isolated Ledgers)

To prevent group-fund confusion, users can log items under a "Personal Pocket." These expenses (e.g., personal souvenirs, specific diets) are tracked solely for the individual's personal trip budget and are completely ignored by the group debt settlement algorithm.

1.4 Real-Time Visibility

- **The Live Feed:** A chronological, trip-wide feed (powered by WebSockets/Socket.io) that broadcasts every transaction, deposit, and trip event instantly to all participants, ensuring maximum transparency.
- **Person-to-Person Ledger:** A detailed, 1-on-1 view showing exactly what User A paid for

User B, what User B paid for User A, and the final net settlement amount between the two.

1.5 Offline-First Capabilities (No Internet Required)

- **Client-Side Timestamping:** The exact time of a transaction is recorded locally via the device (e.g., `new Date().toISOString()`) instead of relying on the database timestamp. This ensures accurate chronological logs in the live feed even when synced later.
- **Local Storage Queue:** When offline, the React app uses IndexedDB (via a wrapper like Dexie.js) to store new expenses locally. The UI updates instantly as if the data was sent.
- **Automatic Sync:** A Service Worker monitors the network status. Once an internet connection is detected, the app automatically dispatches all pending offline transactions to the server in a single batch.

2. Database Schema (MongoDB)

The application will utilize MongoDB. Below are the structural outlines for the primary collections:

```
// Trip Collection
{
  "_id": "ObjectId",
  "name": "String (e.g., 'Goa 2026')",
  "joinCode": "String (Length: 6, Alphanumeric)",
  "groupLead": "ObjectId (Ref: User)",
  "participants": ["ObjectId (Ref: User)"],
  "kittyBalance": "Number (Default: 0)",
  "createdAt": "Timestamp"
}

// Expense Collection
{
  "_id": "ObjectId",
  "tripId": "ObjectId (Ref: Trip)",
  "title": "String",
  "amount": "Number",
  "expenseType": "String (Enum: ['kitty', 'personal'])",
  "paidBy": "ObjectId (Ref: User)",
  "splitAmong": ["ObjectId (Ref: User)"],
  "clientCreatedAt": "String (ISO Date for offline sync)",
  "createdAt": "Timestamp (Server receive time)"
}
```

}

3. 10-Step Development Plan (Progressive Web App)

To deploy simultaneously across Web, Android, and iOS without App Store fees, the optimal tech stack is the **MERN stack engineered as a Progressive Web App (PWA)**.

Step	Phase	Description
1	UI/UX Wireframing	Design the user flows in Figma. Focus on the mobile experience, particularly the offline alert states and the Live Feed timeline.
2	Database Setup	Initialize a MongoDB Atlas cluster. Define Mongoose schemas, ensuring you include <code>clientCreatedAt</code> for offline sync precision.
3	Backend API & Auth	Build the Node.js/Express server. Implement JWT authentication and the logic to generate unique 6-digit PINs.
4	Core Trip Logic (API)	Develop endpoints to handle Kitty transactions and Personal Pocket logging. Create a specific batch-sync endpoint to handle arrays of

Step	Phase	Description
		offline expenses.
5	Settlement Algorithm	Write the logic to calculate net balances and optimize the fewest number of transactions needed to settle debts at the end of the trip.
6	Real-Time Sockets	Integrate Socket.io to the Node backend to broadcast 'newExpense' events to all connected clients.
7	Web Frontend (React)	Build the responsive application using React/Next.js and Tailwind CSS. Implement standard API calls and Socket listeners.
8	Offline & PWA Setup	Integrate IndexedDB (via localforage or Dexie) to cache offline inputs. Configure a Web App Manifest and Service Worker for background syncing and PWA installation.
9	Cloud Deployment	Deploy the backend Node server to Render. Deploy the React PWA front-end to Vercel.

Step	Phase	Description
10	Free App Distribution	Distribute the app by sharing the Vercel link via WhatsApp. Instruct friends to open the link and click "Add to Home Screen" to install it as a native-feeling app on Android and iOS.